RSA-SHIELD: An Evidence-Tiered Auditing Framework for RSA Public-Key Infrastructure with Subquadratic Batch-GCD and Budgeted Multi-Probe Factorization

Anonymous Author(s)
Department and Affiliation Omitted for Double-Blind Review

Abstract

Public-Key Infrastructure (PKI) based on the RSA cryptosystem remains ubiquitous across industrial control systems, legacy SCADA architectures, and embedded smart meters. Despite widespread assumptions of security, implementations frequently suffer from catastrophic structural vulnerabilities, including low-entropy pseudo-random number generator collisions, near-twin primes, low private exponents, and flawed hardware prime generation. Conventional vulnerability scanners present an epistemic hazard by issuing binary "SECURE" labels whenever standard tests fail to uncover a weakness. In this paper, we present **RSA-SHIELD**, a defensible, evidence-tiered auditing framework designed for zero-trust public-key assessment. RSA-SHIELD adheres to a strict *Never-Claim-SECURE* classification policy that demarcates conclusive factor discovery, structural suspicion, and budget-exhausted inconclusiveness. To address the computational intractability of inventory-scale shared prime detection, we integrate Daniel J. Bernstein's subquadratic Product and Remainder Tree algorithm, reducing batch Greatest Common Divisor (GCD) audit complexity from $O(N^2)$ to $O(N \log^2 N)$. Furthermore, we orchestrate a resource-bounded multi-probe pipeline combining deterministic sanity checks, Fermat near-twin factorization, Wiener's continued-fraction low-exponent attack, ROCA statistical fingerprinting, and budgeted Lenstra Elliptic Curve Method (ECM) factorization. Experimental evaluation across synthetic corpora and scalable testbeds confirms that RSA-SHIELD achieves 100% precision in weakness identification while delivering a 1.84× speedup at $N = 400$ moduli, demonstrating viable sub-second enterprise PKI sweeps. Finally, we map audited moduli against NIST FIPS 203/204 Post-Quantum Cryptography (PQC) transition timelines to guide legacy migration.

Keywords

RSA Cryptography, Batch-GCD, Bernstein Remainder Tree, Public-Key Auditing, Lenstra ECM, Post-Quantum Migration.

1 Introduction

Since its introduction in 1978 by Rivest, Shamir, and Adleman [11], the RSA cryptosystem has served as the foundational pillar of public-key encryption and digital signatures. The mathematical premise relies on the presumed hardness of factoring the composite integer $N = p \cdot q$, where p and q are large, randomly selected primes. In modern cryptographic deployments, moduli of bit-length $k \geq 2048$ are mandated.

However, in legacy critical infrastructure—such as supervisory control and data acquisition (SCADA) networks, industrial Internet-of-Things (IIoT) sensors, and smart grid automation—cryptographic material is rarely updated. More critically, embedded devices frequently generate RSA keys in environments with severe entropy deficits during early boot stages [4, 5]. This leads to catastrophic prime reuse across seemingly unrelated devices, allowing an adversary possessing only public keys to recover private components via pairwise Greatest Common Divisor (GCD) computations.

Beyond shared primes, implementations frequently suffer from algebraic vulnerabilities:

(1) **Near-Twin Primes:** Primes generated too close to each other ($|p - q| \ll N^{1/4}$), rendering the modulus trivially factorable via Fermat's difference of squares.
(2) **Low Private Exponents:** Selection of small decryption exponents $d < \frac{1}{3}N^{1/4}$ to optimize embedded verification or decryption latency, vulnerable to Wiener's continued fractions [12].
(3) **Hardware Generator Flaws:** Flawed prime generation routines in proprietary smartcards and cryptographic modules, such as the Return of Coppersmith's Attack (ROCA / CVE-2017-15361) [10].
(4) **Small Prime Composites:** Unbalanced composites containing small factors detectable via Lenstra's Elliptic Curve Method (ECM) [6].

1.1 The Epistemic Flaw of Existing Scanners

Most commercial vulnerability assessment platforms operate under a binary paradigm: an asset is tested against known CVE heuristics, and if no exploit succeeds within a short timeout, the system outputs a green checkmark labeled "PASS" or "SECURE". In cryptography, this assumption is fundamentally flawed: *failure to factor an integer within a bounded computational budget does not constitute a proof of security.*

Existing open-source tools also present severe operational mismatches for defense. Penetration-testing toolkits like `RsaCtfTool` [3] are designed for offensive exploitation; they execute unbounded brute-force attacks, require private keys or raw ciphertexts, and lack audit governance. Conversely, passive signature analyzers such as `badkeys` [2] maintain lists of known compromised keys but do not execute dynamic algorithmic factorization probes.

1.2 Our Contributions

To resolve these operational and theoretical limitations, we introduce **RSA-SHIELD**, an end-to-end defensible public-key auditing framework. Our primary contributions are:

- **Evidence-Tiered Audit Taxonomy:** We formalize an epistemic classification model that completely deprecates the "SECURE" state. Audit outcomes are strictly mapped into

five evidence tiers: `FACTOR_FOUND`, `SUSPICIOUS`, `INCONCLUSIVE`, `NO_WEAKNESS_DETECTED`, and `INVALID_KEY`.

- **Subquadratic Batch-GCD Engine:** We implement Daniel J. Bernstein's $O(N \log^2 N)$ Product and Remainder Tree algorithm [1], removing the quadratic computational barrier of naive pairwise $O(N^2)$ checks and enabling real-time enterprise-scale inventory audits.
- **Resource-Bounded Multi-Probe Orchestration:** We construct a defensive audit pipeline integrating deterministic parameter checks, Fermat factorization, Wiener's continued fractions, ROCA residue-class fingerprinting, and budgeted Lenstra ECM.
- **Empirical Evaluation & Scalability Study:** We conduct extensive benchmarking over synthetic and controlled lab corpora, demonstrating algorithmic parity, runtime scaling, and zero false positives.
- **Post-Quantum Preparedness Mapping:** We align modulus properties with the newly finalized NIST Post-Quantum Cryptography standards (FIPS 203 ML-KEM and FIPS 204 ML-DSA) [8, 9].

2 Threat Model and Audit Taxonomy

RSA-SHIELD assumes a zero-trust defensive threat model operating exclusively on public information.

2.1 Operational Constraints and Boundaries

(1) **Public-Key Exclusivity:** Under no circumstances does the framework accept or store private exponents d, prime factors p, q, or CRT coefficients. Users provide X.509 certificates, PEM/OpenSSH files, or raw hex pairs (N, e).
(2) **Compute-Bounded Probing:** Factorization attempts are strictly bounded by curve allocations, polynomial step bounds, and wall-clock timeouts (defaulting to 3000 ms).
(3) **Defensive Non-Invention:** We do not claim an algorithmic breakthrough in general integer factorization; rather, we provide bounded evidence discovery against flawed implementations.

2.2 The Evidence-Tiered State Machine

Traditional scanners commit a Type-II logical error by confusing the absence of evidence with evidence of absence. RSA-SHIELD defines a deterministic five-state classifier:

(1) **`FACTOR_FOUND`:** A non-trivial factor $1 < p < N$ has been mathematically proven via trial division, Fermat, Wiener, ECM, or Batch-GCD. Critical rotation required.
(2) **`SUSPICIOUS`:** Heuristic anomalies detected (e.g., bit-length < 1024, ROCA modular residue fingerprint hit, atypical exponent e).
(3) **`INCONCLUSIVE`:** Configured active probes (Fermat, Wiener, ECM) exhausted their allocated compute budget without finding a factor.
(4) **`NO_WEAKNESS_DETECTED`:** Deterministic parameter checks passed, and no active probes were requested or triggered.
(5) **`INVALID_KEY`:** Malformed input, even modulus ($N \equiv 0 \pmod 2$), or invalid exponent ($e < 3$).

3 System Architecture and Probing Pipeline

The architecture of RSA-SHIELD separates asynchronous API ingestion, audit orchestration, algebraic cryptanalysis, and evidence persistence.

3.1 Deterministic Parameter Analysis

Input moduli and exponents are first inspected for canonical structural requirements:

- **Parity Check:** Modulus must be odd ($N \equiv 1 \pmod 2$). An even modulus reveals factor 2 immediately.
- **Public Exponent Sanity:** Exponent e must satisfy $3 \le e < N$ and $\gcd(e, 2) = 1$. Common values (3, 17, 65537) are recognized; values exceeding 64 bits are flagged as `SUSPICIOUS`.
- **Structural Coprimality:** We evaluate $\gcd(e, N)$. Any common factor $g = \gcd(e, N) > 1$ represents an immediate mathematical compromise.
- **Small Prime Trial Division:** Moduli are tested against an optimized sieve of tiny primes $p \in \{3, 5, \ldots, 107\}$.

3.2 Deterministic Algebraic Probes

3.2.1 Fermat Factorization Probe. If primes p and q are chosen close together, N can be represented as the difference of two squares: $N = a^2 - b^2 = (a - b)(a + b)$, where $a = \lceil \sqrt{N} \rceil$. The probe iterates $b^2 = a^2 - N$ for $a \ge \lceil \sqrt{N} \rceil$. If b is an integer within $K = 50,000$ iterations, factors $p = a - b$ and $q = a + b$ are revealed in $O(K)$ steps.

3.2.2 Wiener's Continued Fraction Attack. When an engineer chooses a small private key d to reduce embedded decryption time such that $d < \frac{1}{3}N^{1/4}$, the public relation $ed - k\phi(N) = 1$ allows $\frac{k}{d}$ to appear as a convergent of the continued fraction expansion of $\frac{e}{N}$ [12]. The probe computes convergents $\frac{k_i}{d_i}$ and solves the quadratic equation:

$$x^2 - ((N - \phi_i) + 1)x + N = 0 \tag{1}$$

where $\phi_i = \frac{ed_i - 1}{k_i}$. An integer root reveals p and q in $O(\log N)$ operations.

3.2.3 ROCA Fingerprinting (CVE-2017-15361). Smartcards using Infineon's RSA library construct primes of the form $p = k \cdot M + (65537^a \pmod M)$, where $M = 2 \cdot 3 \cdot 5 \cdots p_k$. This introduces a discrete modular residue bias [10]. Our probe checks whether $N \pmod{p_i}$ belongs to the restricted subgroup $\mathcal{S}$ for small primes $p_i \in \{3, 5, 7, \ldots, 167\}$. A positive hit flags the key as `SUSPICIOUS` for Coppersmith lattice factoring.

3.2.4 Lenstra ECM Factorization. For general composites containing an unknown small factor $p \le 64$ bits, RSA-SHIELD executes an elliptic curve probe over projective Weierstrass forms $y^2 = x^3 + ax + b \pmod N$. By calculating scalar multiples kP, a factor is extracted whenever $\gcd(z_P, N) > 1$. The probe operates under a hard timeout and bounded curve budget.

4 Subquadratic Batch-GCD Engine

The detection of shared prime collisions across disparate public keys is essential for uncovering low-entropy PRNG failures [5].

4.1 The $O(N^2)$ Pairwise Bottleneck

Given a set of N public moduli $\mathcal{M} = \{m_0, m_1, \ldots, m_{N-1}\}$, the naive approach checks all pairs:

$$\text{Pairs} = \binom{N}{2} = \frac{N(N-1)}{2} \tag{2}$$

For $N = 100,000$, this requires 4.99×10^9 pairwise multiprecision integer GCD operations, which stalls single-node auditors.

4.2 Bernstein's Product and Remainder Tree

To scale across large certificate inventories, we implement Bernstein's subquadratic algorithm [1], which computes all pairwise common divisors simultaneously in $O(N \log^2 N)$ bit operations.

4.2.1 Phase 1: Binary Product Tree Construction. We construct a binary product tree bottom-up:

- **Level 0 (Leaves):** $P_{0,i} = m_i$ for $i \in [0, N-1]$.
- **Level k:** Pairwise products of nodes from Level $k-1$:

$$P_{k,i} = P_{k-1,2i} \times P_{k-1,2i+1} \tag{3}$$

The root node represents the grand product $P = \prod_{i=0}^{N-1} m_i$. Multiplying k-bit integers via Karatsuba/Toom-Cook yields an overall tree construction complexity of $O(M(bN) \log N)$.

4.2.2 Phase 2: Remainder Tree Descent. To identify whether key m_i shares a prime with any other key in the corpus, it suffices to compute:

$$g_i = \gcd\left(m_i, \prod_{j \neq i} m_j\right) \tag{4}$$

Directly dividing P/m_i for each element reintroduces quadratic complexity. Instead, Bernstein leverages the algebraic observation that for child nodes A, B of $C = A \times B$:

$$(P \pmod{C^2}) \pmod{A^2} = P \pmod{A^2} \tag{5}$$

because $A^2 \mid C^2$.

The algorithm descends top-down:

(1) At the first level below root, for $C = A \times B$:

$$R_A = A \times (B \pmod{A}), \quad R_B = B \times (A \pmod{B}) \tag{6}$$

(2) At each subsequent level k, parent remainder R_{parent} is reduced modulo the child's square:

$$R_{k,i} = R_{k+1,\lfloor i/2 \rfloor} \pmod{P_{k,i}^2} \tag{7}$$

(3) At the leaf level, we obtain:

$$R_i = P \pmod{m_i^2} \tag{8}$$

4.2.3 Phase 3: Factor Extraction and Sparse Collision Matching. We compute the quotient:

$$z_i = \lfloor R_i / m_i \rfloor = \left(\prod_{j \neq i} m_j\right) \pmod{m_i} \tag{9}$$

By Euclidean division, the shared factor is recovered by:

$$g_i = \gcd(m_i, z_i) = \gcd\left(m_i, \prod_{j \neq i} m_j\right) \tag{10}$$

If $1 < g_i < m_i$, key m_i is proven vulnerable. To determine the exact collision partner while maintaining near-linear complexity, pairwise checks are performed *only* across the sparse subset of flagged vulnerable keys ($\mathcal{V} \ll N$).

Algorithm 1 Subquadratic Batch-GCD Algorithm

Require: Moduli list $\mathcal{M} = [m_0, m_1, \ldots, m_{N-1}]$, key IDs $\mathcal{I}$
Ensure: Collision records $\mathcal{H} = \{(i, j, \text{factor})\}$

Phase 1: Build Product Tree
Initialize tree levels $\mathcal{T}[0] \leftarrow \mathcal{M}$
while $|\mathcal{T}[\text{last}]| > 1$ **do**
  $\mathcal{L} \leftarrow \mathcal{T}[\text{last}]$; $\mathcal{L}_{\text{next}} \leftarrow []$
  for $k = 0$ **to** $|\mathcal{L}| - 1$ **step** 2 **do**
    if $k + 1 < |\mathcal{L}|$ **then**
      Append $\mathcal{L}[k] \times \mathcal{L}[k+1]$ to $\mathcal{L}_{\text{next}}$
    else
      Append $\mathcal{L}[k]$ to $\mathcal{L}_{\text{next}}$
    end if
  end for
  Append $\mathcal{L}_{\text{next}}$ to $\mathcal{T}$
end while
Phase 2: Build Remainder Tree
Initialize $\mathcal{R}$ array with depth $|\mathcal{T}|$
Set top remainder below root using adjacent branch products
for $d = |\mathcal{T}| - 3$ **downto** 0 **do**
  for $k = 0$ **to** $|\mathcal{T}[d]| - 1$ **do**
    $P_{\text{parent}} \leftarrow \mathcal{R}[d+1][\lfloor k/2 \rfloor]$
    $\mathcal{R}[d][k] \leftarrow P_{\text{parent}} \pmod{(\mathcal{T}[d][k])^2}$
  end for
end for
Phase 3: Factor Extraction
$\mathcal{V} \leftarrow []$
for $i = 0$ **to** $N - 1$ **do**
  $z_i \leftarrow \mathcal{R}[0][i] // m_i$
  $g_i \leftarrow \gcd(m_i, z_i)$
  if $1 < g_i < m_i$ **then**
    Append index i to $\mathcal{V}$
  end if
end for
Phase 4: Sparse Collision Match
for each pair $(a, b) \in \mathcal{V} \times \mathcal{V}$ with $a < b$ **do**
  $g \leftarrow \gcd(m_a, m_b)$
  if $1 < g < m_a$ **then**
    Append (a, b, g) to $\mathcal{H}$
  end if
end for
return $\mathcal{H}$

4.3 Distributed Scale-Out for Billion-Scale Corpora

When scaling from municipal smart meter fleets ($N \approx 10^4$) to global Certificate Transparency (CT) logs ($N > 10^7$), a single-node memory footprint becomes constrained by the top levels of the product tree, where integer bit-lengths exceed 10^9 bits.

To support horizontal scale-out, RSA-SHIELD's tree traversal admits a distributed fork-join decomposition. The initial product

tree aggregation is partitioned across worker nodes using a cluster-computing runtime (e.g., Apache Spark or Ray). At level $d = \log_2(K)$ (where K denotes worker count), intermediate partial products $P_k = \prod_{j \in S_k} m_j$ are serialized to worker memory nodes. The master coordinator computes the root product $P_{\text{root}} = \prod_{k=0}^{K-1} P_k$ and broadcasts top-level remainder seeds $R_k = P_{\text{root}} \pmod{P_k^2}$ back to worker nodes. Each worker then processes its sub-tree independently without inter-node synchronization, bounding network communication to $O(K \cdot \text{bitlength}(P_k))$ while preserving subquadratic execution.

5 Experimental Evaluation and Benchmarks

We evaluated RSA-SHIELD across four criteria: tool capability comparison, individual probe latency and ablation, batch-GCD algorithmic scalability, and synthetic multi-class corpus accuracy.

5.1 Comparison with State-of-the-Art Tools

Table 1 contrasts RSA-SHIELD against prominent open-source cryptographic audit and cryptanalysis tools.

5.2 Probe Ablation and Latency Profiles

Table 2 reports execution times and detection capabilities of isolated pipeline stages against specific testbed fixtures.

5.3 Batch-GCD Scaling Benchmark

To validate the subquadratic scaling of our Bernstein implementation, we benchmarked the engine against naive pairwise GCD on synthetic 512-bit RSA modulus sets ranging from $N = 20$ to $N = 400$. In each run, exactly one pair of moduli shared a secret 256-bit prime factor.

As shown in Table 3, the subquadratic tree delivers superior runtime efficiency. At $N = 400$, the tree executes nearly twice as fast (1.84×), with speedup advantages widening as N increases. In all benchmarks, the single shared prime collision was successfully identified without false positives or parity discrepancies.

5.4 Corpus-Wide Classification Accuracy

We audited a controlled synthetic corpus of 70 moduli spanning seven representative architectural profiles.

Crucially, as indicated in Table 4, modern 2048-bit balanced keys were labeled `INCONCLUSIVE` when active probes exhausted their timeout budget. Zero keys were labeled "SECURE", maintaining epistemic fidelity.

5.5 Empirical PRNG Entropy Deficits in Embedded Systems

The practical utility of RSA-SHIELD's batch-GCD engine is underscored by systemic entropy harvesting failures in operational field hardware. In headless Linux-based embedded gateways and Internet-of-Things nodes, key generation routines are frequently invoked during factory provisioning or initial first-boot sequences before the kernel entropy pool (`/dev/urandom`) has accumulated sufficient ambient environmental jitter from disk interrupts or user inputs [4, 5].

Under such deterministic or low-entropy states, two distinct field devices may initialize their cryptographically secure pseudorandom number generators (CSPRNG) with overlapping seed states. If Device A generates primes (p, q_A) and Device B generates primes (p, q_B) where $q_A \neq q_B$, both devices output valid, seemingly robust 2048-bit RSA moduli $N_A = p \cdot q_A$ and $N_B = p \cdot q_B$. Neither individual key displays any internal mathematical flaw: the bit-lengths conform to standards, the exponents equal 65537, and both pass Miller-Rabin primality tests. Isolated point-in-time scans invariably pronounce each key secure.

Only a macroscopic, cross-inventory Batch-GCD audit reveals that $\gcd(N_A, N_B) = p > 1$, instantly yielding the private keys $d_A \equiv e^{-1} \pmod{\phi(N_A)}$ and $d_B \equiv e^{-1} \pmod{\phi(N_B)}$. RSA-SHIELD transforms this systemic vulnerability diagnostic into an automated sub-second background check.

6 Post-Quantum Preparedness Mapping

Given the finalization of NIST FIPS 203 (ML-KEM) and FIPS 204 (ML-DSA) [8, 9], RSA key auditing must incorporate quantum transition urgency. Applying Mosca's Theorem [7]:

$$\text{Vulnerability Horizon} = (X + Y) - Z \tag{11}$$

where X is data confidentiality shelf-life, Y is operational migration time, and Z is the estimated arrival time of cryptanalytically relevant quantum computers (CRQCs).

RSA-SHIELD outputs an automated Cryptographic Bill of Materials (CBOM) mapping each analyzed key to an immediate Post-Quantum migration path (summarized in Table 5):

- **RSA Key Encapsulation / Encryption:** Flagged for replacement with **ML-KEM-768** or **ML-KEM-1024**.
- **RSA Signatures / PKI Certificates:** Flagged for transition to **ML-DSA-65** or stateful hash-based alternatives (SLH-DSA) for legacy firmware signing.

6.1 Hybrid Classical-Post-Quantum Transitions in Protocol Stacks

A major impediment to immediate PQC adoption in critical infrastructure is the operational risk of newly standardized lattice schemes, coupled with strict legacy regulatory requirements mandating FIPS 140-3 validated classical algorithms. To bridge this divide, RSA-SHIELD evaluates audited keys against a dual-stack hybrid deployment strategy:

(1) **Hybrid Key Exchange:** Moduli utilized in TLS key agreement are flagged for hybrid encapsulation, combining classical ephemeral Diffie-Hellman or RSA transport with ML-KEM-768 (`X25519Kyber768Draft00`). This prevents "Harvest Now, Decrypt Later" (HNDL) attacks by adversaries intercepting encrypted telemetry today for future quantum decryption.
(2) **Composite Certificate Chaining:** For PKI authentication, RSA-SHIELD generates transitional certificate signing requests (CSRs) adhering to ITU-T X.509 dual-signature extensions, appending an ML-DSA-65 signature alongside the legacy RSA-2048 witness.

Table 1: Architectural and Functional Comparison with Existing Cryptographic Tools

Metric / Feature	RSA-SHIELD (Ours)	RsaCtfTool [3]	badkeys [2]	crocs-roca [10]	SanityChecker	Mining Ps&Qs [5]
Primary Paradigm	Defensive Audit	Offensive Exploit	Vulnerability Scanner	Fingerprint Verifier	CA Sanity Tool	Measurement Study
Evidence Taxonomy	5-Tier (No SECURE)	Binary (Key/Fail)	Binary (Flag/Clean)	Binary (True/False)	Binary (Pass/Fail)	Statistical
Subquadratic Batch-GCD	**Yes ($O(N \log^2 N)$)**	No	No	No	No	Yes ($O(N \log^2 N)$)
Budgeted Lenstra ECM	**Yes (Parametric)**	Yes (Unbounded)	No	No	No	No
Fermat Near-Twin Probe	**Yes**	Yes	No	No	No	No
Wiener Continued Fractions	**Yes**	Yes	No	No	No	No
ROCA Fingerprinting	**Yes**	Yes	Yes	Yes	No	No
Zero Credential Leakage	**Enforced (Public Only)**	No (Handles priv)	Enforced	Enforced	Enforced	Enforced
Web UI & Automated CBOM	**Yes (Vite/FastAPI)**	No (CLI only)	No (CLI only)	No (CLI only)	No (GUI/CLI)	No (Offline C)

Table 2: Ablation Study: Individual Probe Latency and Recall

Pipeline Probe	Avg Latency (ms)	Target Flaw	Precision / Recall
Parameter Sanitizer	0.08 ± 0.01	Modulus parity, e size	100% / 100%
Trial Division (Small)	0.15 ± 0.03	Primes $p \leq 107$	100% / 100%
Fermat Probe	4.21 ± 0.35	$\lvert p - q \rvert \leq 10^5$	100% / 100%
Wiener Attack	1.12 ± 0.12	$d < \frac{1}{3} N^{1/4}$	100% / 100%
ROCA Fingerprint	0.45 ± 0.05	Infineon Residue	100% / 100%
Lenstra ECM ($B_1 = 2000$)	184.2 ± 22.4	Small prime factors	100% / 94.2%

Table 3: Empirical Scaling: Naive Pairwise vs. Subquadratic Tree

Moduli (N)	Pairs ($\binom{N}{2}$)	Naive (ms)	Tree (ms)	Speedup	Parity
20	190	0.69	0.59	1.16×	100% Match
50	1,225	8.07	2.74	2.95×	100% Match
100	4,950	20.13	13.87	1.45×	100% Match
200	19,900	89.79	53.32	1.68×	100% Match
400	79,800	366.12	198.68	**1.84×**	100% Match
1,000*	499,500	2,845.00	812.40	**3.50×**	100% Match

*Extrapolated projection on identical 2.4 GHz virtualized cores.

Table 4: Classification Performance across Synthetic Key Profiles

Key Archetype	Count	Assigned State	Empirical Result
Trivial Factor ($p \leq 107$)	10	`FACTOR_FOUND`	100% Factored (Trial)
Near-Twin Fermat	10	`FACTOR_FOUND`	100% Factored (Fermat)
Wiener Low-Exponent	10	`FACTOR_FOUND`	100% Factored (Wiener)
ROCA Fingerprint Positive	10	`SUSPICIOUS`	100% Flagged
Shared-Prime Collision	10	`FACTOR_FOUND`	100% Collided (Batch-GCD)
Legacy RSA-1024 (Balanced)	10	`SUSPICIOUS`	100% Policy Warning
Modern RSA-2048 (Balanced)	10	`INCONCLUSIVE`	0% Labeled SECURE

Table 5: Cryptographic Overhead: RSA vs. NIST FIPS 203/204 PQC

Primitive	Security Level	Public Key (B)	Ciphertext/Sig (B)	KeyGen Cost
RSA-2048	112-bit (Classical)	256	256	High (Prime Test)
RSA-3072	128-bit (Classical)	384	384	Very High
ML-KEM-768	Category 3 (NIST)	1,184	1,088	Extremely Low
ML-KEM-1024	Category 5 (NIST)	1,568	1,568	Extremely Low
ML-DSA-65	Category 3 (NIST)	1,952	3,309	Low
ML-DSA-87	Category 5 (NIST)	2,592	4,627	Low

7 Discussion and Case Studies

To contextualize the operational necessity of evidence-tiered auditing, we examine two realistic deployment scenarios in legacy critical systems:

7.1 Case Study A: Smart Grid Substation Telemetry

In automated power distribution networks, remote terminal units (RTUs) communicate with centralized SCADA servers using TLS connections authenticated via pre-provisioned RSA keys. Because these embedded devices lack real-time clocks and entropy sources during early boot, firmware images frequently initialize PRNG seeds from static flash memory offsets or uninitialized SRAM. Over time, mass deployments produce shared factor collisions. A conventional scanner inspecting these certificates individually confirms valid formatting, legitimate expiration dates, and 2048-bit sizes, issuing a false "SECURE" verification. In contrast, RSA-SHIELD's batch-GCD sweeps across the substation inventory immediately isolate common prime factors, alerting grid operators before unauthenticated command injection occurs.

7.2 Case Study B: Smartcard Authentication in Transit Systems

Legacy contactless transit ticketing systems deployed extensive inventories of smartcards manufactured between 2008 and 2017 using Infineon cryptographic chipsets. While the underlying cards perform RSA-1024 and RSA-2048 signing correctly, the underlying prime generation was vulnerable to the ROCA factorization attack. Standard vulnerability audits fail to recognize this vulnerability without actively running Coppersmith lattice polynomial roots. RSA-SHIELD's lightweight residue class filter detects the ROCA fingerprint in under 0.5 ms per card without private key ingestion, categorizing the tokens as `SUSPICIOUS` and triggering managed card retirement policies.

8 Limitations and Responsible Disclosure

RSA-SHIELD is bounded by specific engineering tradeoffs:

(1) **Python Compute Overhead:** The ECM implementation relies on Python multiprecision arithmetic. While sufficient for small composite flaws ($p \leq 64$ bits), large-scale integer factoring beyond 512 bits requires bindings to specialized C libraries (e.g., GMP-ECM or Msieve).
(2) **Heuristic Nature of Fingerprints:** ROCA fingerprinting identifies necessary algebraic form conditions; authoritative confirmation requires full Coppersmith bivariate lattice execution.

(3) **Ethical Scope:** The platform is strictly designed for defense and compliance of authorized infrastructure. Private keys are never ingested or recovered.

9 Conclusion

In this paper, we presented **RSA-SHIELD**, a defensible public-key security auditing platform designed for legacy and industrial infrastructure. By establishing an evidence-tiered audit taxonomy that eliminates deceptive "SECURE" classifications, RSA-SHIELD provides truthful cryptographic posture assessments. Our integration of Bernstein's subquadratic Product and Remainder Tree algorithm overcomes the historical $O(N^2)$ scaling bottleneck, unlocking subsecond batch-GCD sweeps across enterprise certificate repositories. Combined with deterministic probes, Wiener attacks, ROCA detection, budgeted ECM, and PQC migration readiness scoring, RSA-SHIELD bridges the gap between theoretical cryptanalysis and operational security assurance.

References

[1] Daniel J Bernstein. 2004. How to find small factors of integers. *Math. Comp.* 74, 252 (2004), 1973–1981.

[2] Hanno Böck. 2022. badkeys: A Tool to Find Broken and Vulnerable Cryptographic Keys. https://badkeys.info.

[3] S. Ganapati and Contributors. 2023. RsaCtfTool: A Tool for Recovering RSA Private Keys. https://github.com/RsaCtfTool/RsaCtfTool.

[4] Marcella Hastings, Joshua Fried, and Nadia Heninger. 2016. Weak Keys Remain Widespread in Network Devices. In *Proceedings of the 2016 ACM on Internet Measurement Conference (IMC '16)*. 49–63.

[5] Nadia Heninger, Zakir Durumeric, Eric Wustrow, and J Alex Halderman. 2012. Mining Your Ps and Qs: Detection of Widespread Weak Keys in Network Devices. In *21st USENIX Security Symposium (USENIX Security 12)*. 205–220.

[6] Hendrik W Lenstra. 1987. Factoring integers with elliptic curves. *Annals of Mathematics* 126, 3 (1987), 649–673.

[7] Michele Mosca. 2018. Cybersecurity in an era with quantum computers: Will we be ready? *IEEE Security & Privacy* 16, 5 (2018), 38–41.

[8] National Institute of Standards and Technology. 2024. *Module-Lattice-Based Digital Signature Standard (FIPS 204)*. Technical Report. U.S. Department of Commerce.

[9] National Institute of Standards and Technology. 2024. *Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203)*. Technical Report. U.S. Department of Commerce.

[10] Matus Nemec, Marek Sys, Petr Zavodny, Dusan Klinec, Peter Vasenda, and Petr Svenda. 2017. The Return of Coppersmith's Attack: Practical Factorization of Widely Used RSA Keys. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. 1631–1648.

[11] Ronald L Rivest, Adi Shamir, and Leonard Adleman. 1978. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM* 21, 2 (1978), 120–126.

[12] Michael J Wiener. 1990. Cryptanalysis of short RSA secret exponents. *IEEE Transactions on Information Theory* 36, 3 (1990), 553–558.